produces:

```
99
```

The call to new on the second-to-last line creates a new object of class Test, assigning that object to the variable t. Then, on the next line, we call the method t.one. To execute this call, Ruby sets self to t and then looks in t's class for the method one. Ruby finds the method defined in the class and calls it.

Inside the method, we set the instance variable @var to 99. This instance variable will be associated with the current object. What is that object? Since the call to t.one set self to t, within that call of the one method, self will be that particular instance of class Test.

On the next line, the method one calls the method two. Because there's no explicit receiver, self is not changed. When Ruby looks for the method two, it looks in Test, the class of t.

The method two references an instance variable @var. Again, Ruby looks for this variable in the current object and finds the same variable that was set by the method one.

The call to puts inside the two method works the same way. Again, because there's no explicit receiver, self will be unchanged. Ruby looks for the puts method in the class of the current object but can't find it. It then looks in Test's superclass, class Object. Again, it doesn't find puts. However, Object mixes in the module Kernel. We'll talk more about this later; for now we can say that mixed-in modules act as if they were superclasses. The Kernel module *does* define puts, so the method is found and executed.

After two and one return, Ruby resets self to the value it had before the original call to t.one. The code is at the top level, the top level self is an object called main, which is placed there by the Ruby runtime.

This explanation may seem labored, but understanding it is an important part of mastering metaprogramming in Ruby.

Class Definitions and self

We've seen that calling a method with an explicit receiver changes self. Perhaps surprisingly, self is also changed inside a class or module definition. This is a consequence of the fact that class definitions are actually executable code in Ruby—if we can execute code, we need to have a current object. A simple test shows what this object is:

```
puts "Before the class definition"
puts "self = #{self}\n"
class Test
  puts "In the definition of class Test"
  puts "self = #{self}"
  puts "Class of self = #{self.class}\n"
end
puts "After the class definition"
puts "self = #{self}"
```

produces:

```
Before the class definition
self = main
In the definition of class Test
self = Test
Class of self = Class
After the class definition
```